Chapter 5: First-Order Methods

Algorithms for Optimization, 2nd ed. (2026)
Kochenderfer & Wheeler

Lecture Notes

June 13, 2026

Table of Contents

What Are First-Order Methods? I

First-order methods are optimization algorithms that use the *gradient* of the objective function to decide which direction to move next. The word "first-order" refers to the first derivative (as opposed to second-order methods, which also use the Hessian matrix of second derivatives).

Definition: First-Order Method

A first-order method selects its next iterate using only $\nabla f(\mathbf{x})$ (nabla f of $\mathbf{x}$), the vector of all first partial derivatives of f evaluated at the current point $\mathbf{x}$.

Why use the gradient? The gradient $\nabla f(\mathbf{x})$ (read "nabla f at $\mathbf{x}$") points in the direction of *steepest ascent* of f. Moving in the *opposite* direction therefore decreases f as fast as the local linear approximation allows. This is an immediate, geometric insight that drives an entire family of practical algorithms.

Computational advantage. Computing ∇f requires $O(n)$ work for most structured functions (e.g. neural networks via backpropagation), where n is the number of parameters. By contrast, computing the Hessian requires $O(n^2)$ storage alone, making it infeasible for problems with millions of parameters. This is why virtually all modern deep-learning systems rely on first-order methods.

What Are First-Order Methods? II

Requirement

The objective f must be *differentiable* at every point visited by the algorithm (or at least *sub-differentiable* for non-smooth objectives such as the ℓ_1 norm).

What Are First-Order Methods? III

Chapter Roadmap

This chapter develops nine first-order methods in order of increasing sophistication. Each one addresses a specific weakness of the previous method:

1. **Gradient Descent** — the simplest possible first-order method; the foundation for everything else.
2. **Conjugate Gradient** — avoids the zigzag behaviour of gradient descent on elongated functions.
3. **Momentum (Heavy Ball)** — adds velocity to accelerate progress through flat regions.
4. **Nesterov Momentum** — evaluates the gradient one step ahead to reduce overshooting.
5. **AdaGrad** — adapts the step size dimension-by-dimension based on gradient history.
6. **RMSProp** — fixes AdaGrad's vanishing step size using an exponential moving average.
7. **Adadelta** — eliminates the global learning rate entirely using a ratio of two moving averages.
8. **Adam** — combines momentum with adaptive learning rates; the standard choice for deep learning.
9. **Hypergradient Descent** — applies gradient descent to the step size itself, learning it on the fly.

What Are First-Order Methods? IV

The key takeaway from this chapter is that there is no single "best" first-order method: the right choice depends on the structure of the problem, the availability of gradient information, and computational constraints.

5.1 Gradient Descent — Motivation and Setup I

Gradient descent is the conceptual starting point for all first-order optimization. The idea is simple: at each iteration, compute the gradient of f at the current point, then take a step in the opposite direction (the direction of steepest descent).

Notation for iterates. We write $\mathbf{x}^{(k)}$ (read "$\mathbf{x}$ superscript k") to mean the value of the decision variable vector at iteration number k.

Gradient at the k-th Iterate

$$\mathbf{g}^{(k)} = \nabla f\left(\mathbf{x}^{(k)}\right)$$

Here $\mathbf{g}^{(k)}$ (bold g superscript k) is the gradient vector evaluated at the current point $\mathbf{x}^{(k)}$. It lives in $\mathbb{R}^n$ (the space of all real n-dimensional vectors), where n is the number of decision variables. Each component $g_i^{(k)} = \partial f / \partial x_i$ measures how quickly f increases in the i-th coordinate direction.

First-order Taylor approximation. Any smooth function f can be approximated near a point $\mathbf{x}^{(k)}$ by a linear (affine) function. If we move from $\mathbf{x}^{(k)}$ by a small displacement $\alpha\mathbf{d}$ (where α (alpha) is a positive scalar and $\mathbf{d}$ is a unit-length direction vector), the value of f changes approximately as:

5.1 Gradient Descent — Motivation and Setup II

Linear Approximation

$$f\left(\mathbf{x}^{(k)} + \alpha \mathbf{d}\right) \approx f\left(\mathbf{x}^{(k)}\right) + \alpha\, \mathbf{d}^{\top} \mathbf{g}^{(k)}$$

- α (alpha) is the step factor, also called the learning rate; it controls how far we move.
- $\mathbf{d}$ is the direction vector, assumed to have unit length $\|\mathbf{d}\| = 1$ (where $\|\cdot\|$ denotes the Euclidean norm).
- $\mathbf{d}^{\top}\mathbf{g}^{(k)}$ ("d transpose times g") is the dot product of $\mathbf{d}$ and the gradient; it measures how much f increases per unit step in direction $\mathbf{d}$.

How to Read This

The dot product $\mathbf{d}^{\top}\mathbf{g}^{(k)}$ is maximised when $\mathbf{d}$ points in the same direction as $\mathbf{g}^{(k)}$, and minimised (most negative) when $\mathbf{d}$ points in the opposite direction. Therefore, to decrease f as fast as possible, we should choose $\mathbf{d} = -\mathbf{g}^{(k)}/\|\mathbf{g}^{(k)}\|$.

Deriving the steepest descent direction. We want to find the unit vector $\mathbf{d}$ that minimises $\mathbf{d}^\top \mathbf{g}^{(k)}$ subject to $\|\mathbf{d}\| = 1$. By the Cauchy–Schwarz inequality, the minimum value is $-\|\mathbf{g}^{(k)}\|$, achieved when:

Steepest Descent Direction

$$\mathbf{d}^{(k)} = -\frac{\mathbf{g}^{(k)}}{\left\|\mathbf{g}^{(k)}\right\|}$$

This is the direction of steepest descent: it is the negative of the gradient $\mathbf{g}^{(k)}$ (nabla f at the current point), divided by its magnitude $\|\mathbf{g}^{(k)}\|$ so that the result has unit length. Dividing by the norm means α truly controls the *distance* moved, not just an unnormalised step.

The full gradient descent update rule combines the steepest descent direction with the step factor:

5.1 Gradient Descent — Motivation and Setup IV

Gradient Descent Update Rule

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \alpha\,\mathbf{d}^{(k)} = \mathbf{x}^{(k)} - \alpha\,\frac{\mathbf{g}^{(k)}}{\left\|\mathbf{g}^{(k)}\right\|}$$

Many practical implementations omit the normalisation and write $\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} - \alpha\,\mathbf{g}^{(k)}$; in that case α is not the step *length* but a scale factor, and its value must be adjusted accordingly.

Key Insight

Moving in the direction opposite to the gradient guarantees that f decreases (at least for small enough α). This is because locally, the gradient tells us exactly "uphill," so "downhill" is the exact opposite.

5.1 Gradient Descent — Motivation and Setup V

Note on norms. The choice of norm used to define "unit direction" affects which method we recover:

- Using the L_2 (Euclidean) norm $\|\mathbf{d}\|_2 = 1$ gives **steepest descent** (the method above).
- Using the L_1 norm $\|\mathbf{d}\|_1 = 1$ gives **coordinate descent**, where only one component of $\mathbf{x}$ changes per step.

Convergence guarantee. For a function f that is L-smooth (meaning its gradient does not change too quickly; characterised by the Lipschitz constant L (capital el) of the gradient) and μ (mu)-strongly convex (meaning it curves upward with curvature at least $\mu > 0$), gradient descent with the right step size converges at a linear rate:

5.1 Gradient Descent — Motivation and Setup VI

Convergence Rate for Strongly Convex f

$$\left\|\mathbf{x}^{(k)} - \mathbf{x}^*\right\|^2 \leq \left(1 - \frac{\mu}{L}\right)^k \left\|\mathbf{x}^{(1)} - \mathbf{x}^*\right\|^2$$

Here $\mathbf{x}^*$ (x star) is the true minimiser (optimal point), μ (mu) is the strong convexity constant (how curved the function is near the minimum), L is the Lipschitz constant of the gradient (how fast the gradient can change), and k is the iteration number. The ratio $\kappa = \mu/L$ (kappa, called the condition number) determines how fast convergence occurs.

How to Read This

The distance from the optimum shrinks by a factor of $(1 - \mu/L)$ at every step. When $\mu \approx L$ (well-conditioned problem), this factor is close to 0 and convergence is fast. When $\mu \ll L$ (ill-conditioned problem, e.g. a long narrow valley), the factor is close to 1 and convergence is very slow. This is gradient descent's fundamental weakness.

5.1 Gradient Descent — Algorithm I

The gradient descent algorithm is straightforward to state: start somewhere, repeatedly compute the gradient and take a step downhill, and stop after a fixed number of iterations (or when the gradient is small enough to declare convergence).

Algorithm: Gradient Descent (with Normalisation)

Input: objective f, gradient function ∇f, initial point $\mathbf{x}^{(1)} \in \mathbb{R}^n$, step factor $\alpha > 0$ (alpha, the learning rate), number of iterations T (a positive integer).

For $k = 1, 2, \ldots, T$:

1. Evaluate $\mathbf{g}^{(k)} = \nabla f(\mathbf{x}^{(k)})$ (one gradient computation; $O(n)$ cost).
2. Form the descent direction $\mathbf{d}^{(k)} = -\mathbf{g}^{(k)} / \left\|\mathbf{g}^{(k)}\right\|$ (negative normalised gradient).
3. Update the point: $\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \alpha\, \mathbf{d}^{(k)}$.

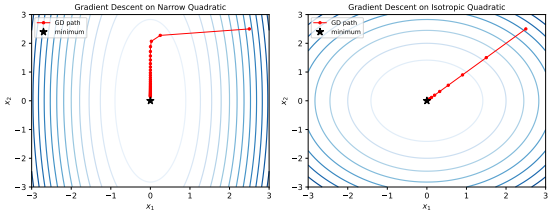
Return $\mathbf{x}^{(T+1)}$ as the best estimate of the minimiser.

Cost analysis. Each iteration costs exactly one gradient evaluation. For most functions this is $O(n)$, making gradient descent practical even when n is in the millions.

5.1 Gradient Descent — Python Code

```python
import numpy as np

def gradient_descent(f, grad_f, x, alpha,
                     n_iter=100, normalize=True):
    """
    f         : objective function
    grad_f    : gradient of f
    x         : initial point (1D numpy array)
    alpha     : step factor (learning rate)
    normalize: if True, use unit step direction
    """
    path = [x.copy()]
    for _ in range(n_iter):
        g = grad_f(x)
        if normalize:
            d = -g / (np.linalg.norm(g) + 1e-15)
        else:
            d = -g           # unnormalized: alpha acts as true scaling
        x = x + alpha * d
        path.append(x.copy())
    return x, np.array(path)

# Example: f(x) = x1^2 * x2
# The gradient is [2*x1*x2, x1^2]
f     = lambda x: x[0]**2 * x[1]
gradf = lambda x: np.array([2*x[0]*x[1], x[0]**2])
```

5.1 Gradient Descent — Paths, Worked Example, and Step Size I



The left panel shows gradient descent on an *ill-conditioned* quadratic (narrow elliptical contours): the iterates zigzag back and forth across the valley, making very slow progress toward the minimum. The right panel shows the same algorithm on a *well-conditioned* (circular) quadratic: the path goes straight to the minimum.

Worked Example: $f(\mathbf{x}) = x_1 x_2^2$

We start at $\mathbf{x}^{(k)} = [1, 2]$ with step $\alpha = 0.1$ and use normalised gradient descent.

Step 1 — compute the gradient. The partial derivatives are $\partial f/\partial x_1 = x_2^2 = 4$ and $\partial f/\partial x_2 = 2x_1 x_2 = 4$. So $\mathbf{g}^{(k)} = [4,\ 4]$.

Step 2 — normalise. The magnitude is $\|\mathbf{g}^{(k)}\| = \sqrt{4^2 + 4^2} = \sqrt{32} \approx 5.66$. The unit descent direction is:

$$\mathbf{d}^{(k)} = -\frac{[4,4]}{\sqrt{32}} = \left[-\frac{1}{\sqrt{2}}, -\frac{1}{\sqrt{2}}\right] \approx [-0.707,\ -0.707]$$

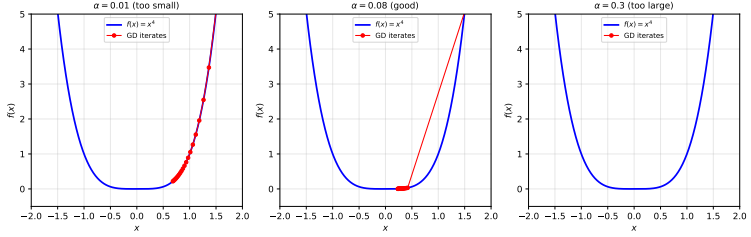
Step 3 — update.

$$\mathbf{x}^{(k+1)} = [1,2] + 0.1 \times [-0.707, -0.707] \approx [0.929,\ 1.929]$$

The function value drops from $f(1,2) = 4$ to

5.1 Gradient Descent — Paths, Worked Example, and Step Size II

Effect of the step size. The step factor α (alpha) must be chosen carefully. The figure below illustrates what happens for three different values:



- **Too small**: the iterates move very slowly toward the minimum; many thousands of iterations may be needed before the change is perceptible.
- **Just right**: the iterates converge smoothly and quickly to the minimum.
- **Too large**: the iterates overshoot the minimum and may diverge entirely, producing larger and larger function values.

5.1 Gradient Descent — Paths, Worked Example, and Step Size III

Key Insight: Gradient Descent's Weakness

On elongated (ill-conditioned) functions, successive gradient directions are nearly perpendicular to each other. Each step undoes part of the progress made by the previous step, producing the characteristic zigzag pattern. The remaining methods in this chapter address this exact problem.

5.2 Conjugate Gradient — Motivation and Conjugacy I

The problem with gradient descent on narrow valleys. On an ill-conditioned quadratic function, the gradient at each step points roughly perpendicular to the valley axis. Consequently, successive steps cancel each other out, and the algorithm zigzags toward the minimum instead of heading straight for it. The *conjugate gradient* (CG) method was designed to fix this.

Quadratic objectives. CG was originally designed to solve the linear system $\mathbf{A}\mathbf{x} = -\mathbf{b}$ exactly, which is equivalent to minimising:

Quadratic Objective

$$\min_{\mathbf{x}\in\mathbb{R}^n} \ \frac{1}{2}\mathbf{x}^\top \mathbf{A}\mathbf{x} + \mathbf{b}^\top \mathbf{x} + c$$

Here $\mathbf{A}$ (bold capital A) is a symmetric, positive definite $n \times n$ matrix (it defines the shape of the bowl), $\mathbf{b}$ (bold b) is a constant vector (it shifts the bowl), and c is a scalar constant (it shifts the function value but does not affect the location of the minimum). When $\mathbf{A}$ is positive definite, the function is strictly convex and has a unique global minimum.

5.2 Conjugate Gradient — Motivation and Conjugacy II

What does "conjugate" mean? In ordinary geometry, two vectors are orthogonal if their dot product is zero. *Conjugacy with respect to* $\mathbf{A}$ is a generalised form of orthogonality that accounts for the shape of the function.

Conjugacy (A-orthogonality)

Directions $\mathbf{d}^{(1)}, \ldots, \mathbf{d}^{(n)}$ are **mutually conjugate** with respect to $\mathbf{A}$ if:

$$\mathbf{d}^{(i)\top} \mathbf{A}\, \mathbf{d}^{(j)} = 0 \quad \text{for all } i \neq j$$

When $\mathbf{A} = \mathbf{I}$ (the identity matrix), this reduces to ordinary orthogonality. For a general positive definite $\mathbf{A}$, conjugate directions are orthogonal in the "stretched" coordinate system defined by $\mathbf{A}$.

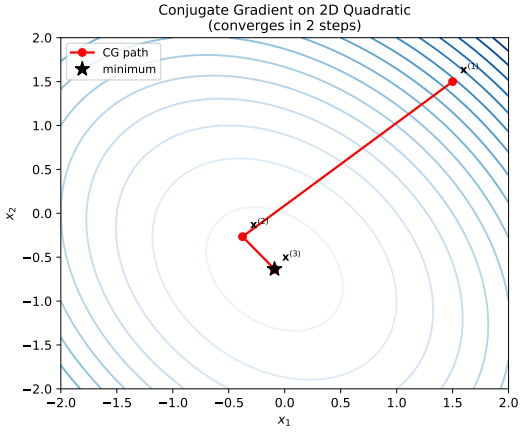
Key Result

If we have n mutually conjugate directions for an n-dimensional quadratic, we can reach the exact minimum in *exactly* n steps, one per direction. No quadratic function in n dimensions requires more than n CG steps.

5.2 Conjugate Gradient — Motivation and Conjugacy III

The figure shows this on a 2-dimensional quadratic ($n = 2$): CG converges in exactly 2 steps, reaching the exact minimum on the second step. Gradient descent, by contrast, would take many more steps due to zigzagging.

Note that the two search directions in the CG path are *not* perpendicular to each other in the usual sense; they are conjugate with respect to the Hessian matrix **A**, meaning they are orthogonal in the warped geometry of the quadratic bowl.



Conjugate gradient converges in $n = 2$ steps on a 2-D quadratic. The directions are conjugate (A-orthogonal) but not geometrically orthogonal.

5.2 Conjugate Gradient — Update Rules I

The CG algorithm builds up a sequence of conjugate directions one at a time, using only information from the current and previous gradients (no matrix $\mathbf{A}$ required for non-quadratic generalisations).

Initialisation. The first step is ordinary steepest descent (there is no previous direction to conjugate against):

$$\mathbf{d}^{(1)} = -\mathbf{g}^{(1)}$$

Exact line search for quadratics. On a quadratic function, the ideal step length can be computed exactly (without any trial-and-error):

Exact Line Search Step Length

$$\alpha^{(k)} = -\frac{(\mathbf{d}^{(k)})^\top \mathbf{g}^{(k)}}{(\mathbf{d}^{(k)})^\top \mathbf{A}\, \mathbf{d}^{(k)}}$$

Here $\alpha^{(k)}$ (alpha superscript k) is the step length at iteration k; it is the unique scalar that minimises f along the line $\mathbf{x}^{(k)} + \alpha \mathbf{d}^{(k)}$. The numerator measures how much f decreases per unit step; the denominator measures the curvature in that direction.

5.2 Conjugate Gradient — Update Rules II

Direction update. After the first step, each new direction is a combination of the current negative gradient and the previous direction:

CG Direction Update

$$\mathbf{d}^{(k)} = -\mathbf{g}^{(k)} + \beta^{(k)}\mathbf{d}^{(k-1)}$$

Here $\beta^{(k)}$ (beta superscript k) is a scalar that controls how much of the previous direction is mixed in. It is chosen to ensure the new direction is conjugate (A-orthogonal) to all previous directions.

5.2 Conjugate Gradient — Update Rules III

Exact β for quadratics (when A is known):

$$\beta^{(k)} = \frac{(\mathbf{g}^{(k)})^\top \mathbf{A}\, \mathbf{d}^{(k-1)}}{(\mathbf{d}^{(k-1)})^\top \mathbf{A}\, \mathbf{d}^{(k-1)}}$$

This requires multiplying vectors by **A**, which is feasible when **A** is known explicitly.

Approximations for general (non-quadratic) functions. When **A** is unknown or when f is not quadratic, several formulas approximate β using only gradient vectors:

Fletcher–Reeves Update

$$\beta_{\mathrm{FR}}^{(k)} = \frac{(\mathbf{g}^{(k)})^\top \mathbf{g}^{(k)}}{(\mathbf{g}^{(k-1)})^\top \mathbf{g}^{(k-1)}}$$

This is the ratio of the squared norm (squared length) of the current gradient to the squared norm of the previous gradient. When the gradient shrinks (good progress), β decreases, blending less of the old direction.

5.2 Conjugate Gradient — Update Rules IV

Polak–Ribière Update

$$\beta_{\mathrm{PR}}^{(k)} = \frac{\left(\mathbf{g}^{(k)}\right)^{\top}\left(\mathbf{g}^{(k)} - \mathbf{g}^{(k-1)}\right)}{\left(\mathbf{g}^{(k-1)}\right)^{\top}\mathbf{g}^{(k-1)}}$$

The numerator now measures how much the gradient has *changed* from the previous step. Convergence can be guaranteed by truncating negative values: $\beta^{(k)} = \max(\beta_{\mathrm{PR}}^{(k)}, 0)$. In practice, Polak–Ribière often outperforms Fletcher–Reeves on non-quadratic functions.

Restarting. For non-quadratic f, the conjugacy property is gradually lost over many iterations. CG is therefore *restarted* every n iterations (or when $(\mathbf{d}^{(k)})^{\top}\mathbf{g}^{(k)} > 0$, meaning the direction is no longer a descent direction): the direction is reset to $\mathbf{d}^{(k)} \leftarrow -\mathbf{g}^{(k)}$.

Key Takeaway

Conjugate gradient extends the power of quadratic solvers to general smooth functions, converging in n exact steps on quadratics and accelerating convergence significantly on well-structured non-quadratic functions.

5.2 Conjugate Gradient — Python Implementation

```python
import numpy as np

def conjugate_gradient(grad_f, x, n_iter=None, restart=True):
    """
    Non-linear conjugate gradient with Fletcher-Reeves beta.
    grad_f  : function returning gradient vector at x
    x       : initial point (numpy array, length n)
    n_iter  : number of iterations (default 10*n)
    restart : reset direction every n steps for non-quadratic f
    """
    n = len(x)
    if n_iter is None:
        n_iter = 10 * n
    g = grad_f(x)
    d = -g.copy()            # first direction: steepest descent
    path = [x.copy()]

    for k in range(1, n_iter + 1):
        # Simple backtracking line search (Armijo condition)
        alpha = 1.0
        while True:
            x_new = x + alpha * d
            g_new = grad_f(x_new)
            # Accept step if gradient norm decreased (proxy check)
            if np.dot(g_new, g_new) < np.dot(g, g):
                break
            alpha *= 0.5
```

5.3 Momentum — Intuition and Update Rule I

The problem momentum solves. In gradient descent, each step depends only on the current gradient and has no memory of previous steps. On functions with flat regions or narrow valleys, the algorithm makes very little progress per step: in flat regions the gradient is tiny, and in narrow valleys successive gradients nearly cancel each other out.

Physical intuition. Imagine a ball rolling down a hilly landscape. Unlike a particle following the instantaneous gradient, the ball *accumulates velocity*: it speeds up in directions where the force (gradient) is consistently pointing the same way, and it damps out oscillations because inertia prevents it from making sharp turns. The momentum method mimics this physical process.

Momentum Update Rule

The method maintains a **velocity vector** $\mathbf{v}^{(k)}$ (bold v superscript k) in addition to the position $\mathbf{x}^{(k)}$:

$$\mathbf{v}^{(k+1)} = \beta\,\mathbf{v}^{(k)} - \alpha\,\mathbf{g}^{(k)}$$

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \mathbf{v}^{(k+1)}$$

where:

- β (beta) is the momentum decay coefficient, chosen in $[0, 1)$; it controls how much of the old velocity is retained. A typical value is $\beta = 0.9$.
- α (alpha) is the step factor (learning rate), which scales how strongly the gradient pushes the velocity.
- $\mathbf{g}^{(k)} = \nabla f(\mathbf{x}^{(k)})$ is the gradient at the current position.
- The velocity is initialised at $\mathbf{v}^{(1)} = \mathbf{0}$.

5.3 Momentum — Intuition and Update Rule III

How to Read This

At each step, the velocity is scaled down by β (like friction) and then pushed by the current gradient. If the gradient keeps pointing in the same direction over many steps, the velocity builds up in that direction. If the gradient oscillates (as in a narrow valley), the oscillations partially cancel in the velocity, producing a smoother trajectory.

5.3 Momentum — Intuition and Update Rule IV

Equivalent heavy ball form. The momentum method can also be written without an explicit velocity variable by substituting $\mathbf{v}^{(k)} = \mathbf{x}^{(k)} - \mathbf{x}^{(k-1)}$:

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} - \alpha\,\mathbf{g}^{(k)} + \beta\Big(\mathbf{x}^{(k)} - \mathbf{x}^{(k-1)}\Big)$$

This is the "heavy ball" form (Polyak, 1964): the third term $\beta(\mathbf{x}^{(k)} - \mathbf{x}^{(k-1)})$ acts like inertia, pulling the next iterate in the direction the last step moved.

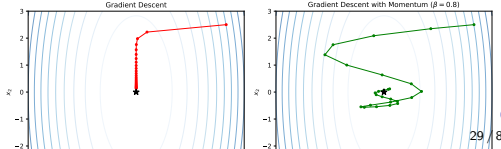
Steady-state velocity. Suppose the gradient were constant at $\mathbf{g}$ (a reasonable approximation in a local neighbourhood). The velocity would converge to a fixed point:

$$\mathbf{v}^* = \frac{-\alpha}{1-\beta}\,\mathbf{g}$$

Comparing to gradient descent (velocity $= -\alpha\mathbf{g}$), the effective step is amplified by a factor of $1/(1-\beta)$. For $\beta = 0.9$ this gives a $10\times$ speed-up in steady state; for $\beta = 0.99$ it gives $100\times$.

Worked Example: Momentum Speed-Up

Consider $f(\mathbf{x}) = 8x_1^2 + x_2^2$ (ill-conditioned, ratio 8:1).
Starting from $\mathbf{x}^{(1)} = [2.5, 2.5]$ with $\alpha = 0.06$, $\beta = 0.8$:
Iteration 1: $\mathbf{g}^{(1)} = [40, 5]$



5.3 Momentum — Intuition and Update Rule V

Key Takeaway

Momentum is most helpful when gradients are consistent over many steps (flat regions, narrow valleys). The parameter β (beta) trades off acceleration against stability: too high and the method overshoots; too low and it barely improves over gradient descent.

5.3 Momentum — Python Implementation

```python
import numpy as np

def momentum_descent(grad_f, x, alpha=0.01, beta=0.9,
                     n_iter=500):
    """
    Gradient descent with momentum (Heavy Ball method).
    grad_f : function returning the gradient vector at x
    x      : initial point (numpy array)
    alpha  : step factor (learning rate), controls gradient influence
    beta   : momentum decay in [0, 1); 0 = pure gradient descent,
             0.9 = strong momentum (typical choice)
    """
    v = np.zeros_like(x)   # velocity initialised to zero
    path = [x.copy()]

    for _ in range(n_iter):
        g = grad_f(x)
        v = beta * v - alpha * g    # scale down old velocity, add gradient push
        x = x + v                   # move to new position
        path.append(x.copy())

    return x, np.array(path)

# -- Example: narrow quadratic f(x) = 4*x1^2 + 0.5*x2^2
#    Condition number = 8 (quite ill-conditioned)
A = np.diag([8.0, 1.0])
grad_f = lambda x: A @ x
```

5.4 Nesterov Momentum — Lookahead Gradient I

The key insight of Nesterov (1983). Standard momentum evaluates the gradient at the *current* position $\mathbf{x}^{(k)}$, but then uses the velocity to move to a *different* position. This is slightly wasteful: we evaluate the gradient at a point we are about to leave. Why not instead evaluate the gradient at the point we are about to *arrive* at?

The lookahead. Before computing the gradient, we first apply the momentum term $\beta\mathbf{v}^{(k)}$ (beta times v superscript k) to get a *lookahead point* $\mathbf{x}^{(k)} + \beta\mathbf{v}^{(k)}$. We evaluate the gradient at this lookahead point, which gives a more accurate estimate of the gradient at the destination.

5.4 Nesterov Momentum — Lookahead Gradient II

Nesterov Update (Algorithm 5.4)

$$\mathbf{v}^{(k+1)} = \beta\,\mathbf{v}^{(k)} - \alpha\,\nabla f\left(\mathbf{x}^{(k)} + \beta\,\mathbf{v}^{(k)}\right)$$

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \mathbf{v}^{(k+1)}$$

The symbols have the same meaning as in standard momentum: $\mathbf{v}^{(k+1)}$ (bold v superscript $k{+}1$) is the new velocity, β (beta) $\in [0, 1)$ is the momentum decay, α (alpha) is the step factor, and $\nabla f(\mathbf{x}^{(k)} + \beta\mathbf{v}^{(k)})$ is the gradient evaluated at the *lookahead* point.

How to Read This

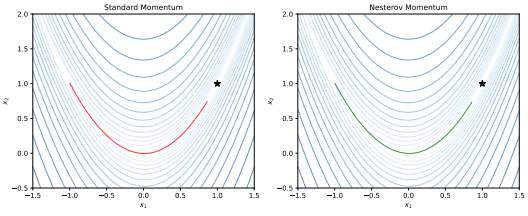
Think of it this way: standard momentum first evaluates where you are, then uses yesterday's momentum to estimate where to go. Nesterov instead *first commits* to moving by the momentum term, then evaluates the gradient at that committed location and adjusts. This "look before you leap" strategy reduces overshooting and improves convergence.

5.4 Nesterov Momentum — Lookahead Gradient III

Convergence improvement. For a convex and smooth function f, standard gradient descent achieves a convergence rate of $O(1/k)$ (the function value error shrinks proportionally to $1/k$). Nesterov momentum achieves $O(1/k^2)$, which is provably optimal for first-order methods on smooth convex functions. This is called an *accelerated* rate.

Practical cost. Nesterov requires one extra gradient evaluation per step compared to standard gradient descent (evaluating at the lookahead point instead of the current point); the asymptotic computational cost is the same.

The figure on the right compares standard momentum and Nesterov momentum on the Rosenbrock function, which has a curved, narrow valley leading to its minimum at $(1, 1)$. Nesterov typically overshoots less at sharp turns and converges in fewer iterations.



Nesterov momentum (right column) typically converges faster on the Rosenbrock function than standard momentum (left column).

5.4 Nesterov Momentum — Lookahead Gradient IV

Key Takeaway

Nesterov momentum is the theoretically optimal first-order method for smooth convex functions. In practice it also works well on non-convex problems (such as training neural networks) and is often preferred over standard momentum when a slightly more careful implementation is acceptable.

5.4 Nesterov Momentum — Python Code

```python
import numpy as np

def nesterov_momentum(grad_f, x, alpha=0.001, beta=0.9,
                      n_iter=500):
    """
    Nesterov accelerated gradient descent.
    grad_f : function returning gradient vector at x
    x      : initial point (numpy array)
    alpha  : learning rate (step factor)
    beta   : momentum decay in [0, 1)

    KEY DIFFERENCE from standard momentum:
    gradient is evaluated at the LOOKAHEAD point x + beta*v,
    not at the current point x.
    """
    v = np.zeros_like(x)
    path = [x.copy()]

    for _ in range(n_iter):
        # Evaluate gradient at lookahead position x + beta*v
        g = grad_f(x + beta * v)
        v = beta * v - alpha * g    # update velocity using lookahead grad
        x = x + v                   # move to new position
        path.append(x.copy())

    return x, np.array(path)
```

5.5 AdaGrad — Adaptive Per-Dimension Step Sizes I

The problem AdaGrad solves. All methods so far use the same step factor α (alpha) for every dimension of $\mathbf{x}$. This is suboptimal when different dimensions have very different gradient magnitudes. For example, in natural language processing, some word features appear very rarely (sparse gradient) while others appear constantly (dense gradient). A uniform step factor either moves too little for the rare features or too much for the common ones.

AdaGrad's idea (Duchi et al., 2011): track the history of squared gradients for each dimension, and use this history to scale the effective step factor. Dimensions with large past gradients get a smaller step; dimensions with small past gradients get a larger step.

5.5 AdaGrad — Adaptive Per-Dimension Step Sizes II

AdaGrad Update

Let $s_i^{(k)}$ (s sub i superscript k) be the accumulated sum of squared gradients for dimension i up to step k:

$$s_i^{(k)} = \sum_{j=1}^{k} \left(g_i^{(j)} \right)^2$$

The **effective step factor** for dimension i is:

$$\alpha_{\text{adagrad},i}^{(k)} = \frac{\alpha}{\varepsilon + \sqrt{s_i^{(k)}}}$$

The **parameter update** is:

$$x_i^{(k+1)} = x_i^{(k)} - \alpha_{\text{adagrad},i}^{(k)} \, g_i^{(k)}$$

5.5 AdaGrad — Adaptive Per-Dimension Step Sizes III

Symbol explanations:

- $s_i^{(k)}$ (s sub i superscript k): the running sum of squared gradients in dimension i up to and including step k. This is a scalar, not a vector. It is always non-negative and never decreases.
- $\sum_{j=1}^{k}$ (Sigma, meaning "sum from $j = 1$ to k"): add up the values for all past iterations.
- $g_i^{(j)}$ (g sub i superscript j): the i-th component of the gradient at step j.
- α (alpha): a global step factor, typically set to 0.01.
- ε (epsilon): a tiny positive constant, typically 10^{-8}, added to prevent division by zero when $s_i^{(k)} = 0$.
- $\sqrt{s_i^{(k)}}$: the square root of the accumulated squared gradient, which has units of gradient magnitude.

5.5 AdaGrad — Adaptive Per-Dimension Step Sizes IV

How to Read This

As a dimension receives more (large) gradient signal over time, s_i grows large and $1/\sqrt{s_i}$ becomes small, reducing that dimension's effective step size. Dimensions that rarely receive gradient signal keep a larger effective step size. AdaGrad automatically identifies and respects these differences without requiring the user to set per-dimension learning rates.

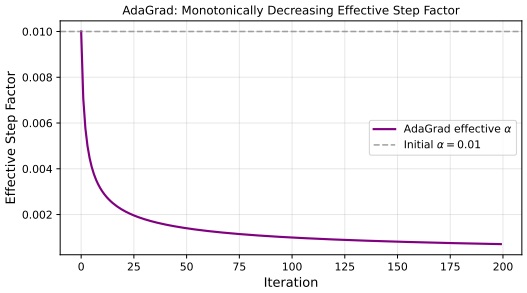
5.5 AdaGrad — Adaptive Per-Dimension Step Sizes V

Worked Example: AdaGrad Step Sizes

Consider $f(\mathbf{x}) = x_1^2 + 10x_2^2$ with $\mathbf{x}^{(1)} = [3.5, 3.5]$ and $\alpha = 0.5$.

Step 1: $\mathbf{g}^{(1)} = [7.0, 70.0]$. $s_1^{(1)} = 49$, $s_2^{(1)} = 4900$. Effective steps: $\alpha_1 = 0.5/\sqrt{49} = 0.5/7 \approx 0.071$, $\alpha_2 = 0.5/\sqrt{4900} = 0.5/70 \approx 0.007$. Dimension 2 already gets a $10\times$ smaller step!

Step 10: After accumulating more squared gradients, s_2 grows much faster than s_1, so dimension 2's step shrinks even further relative to dimension 1.



AdaGrad's effective step factor monotonically decays toward zero over time, which eventually stalls progress on long training runs.

5.5 AdaGrad — Adaptive Per-Dimension Step Sizes VI

AdaGrad's Weakness

Since $s_i^{(k)}$ only ever increases (each new squared gradient is added in), the effective step size $\alpha/\sqrt{s_i^{(k)}}$ can only decrease. For problems where gradients are consistently large (dense gradient tasks like image classification), s_i grows without bound and the effective step eventually becomes negligibly small, stopping learning before convergence. This motivates RMSProp, which is introduced next.

5.5 AdaGrad — Python Implementation

```python
import numpy as np

def adagrad(grad_f, x, alpha=0.01, eps=1e-8, n_iter=500):
    """
    AdaGrad: adapts per-dimension step factors from gradient history.
    grad_f : gradient function
    x      : initial point
    alpha  : global step factor (default 0.01; try 0.1 to 1.0 in practice)
    eps    : small constant to prevent division by zero
    """
    s = np.zeros_like(x)     # s[i] = cumulative sum of squared gradients in dim i
    path = [x.copy()]

    for _ in range(n_iter):
        g = grad_f(x)
        s += g**2                                  # accumulate (s only ever grows)
        x = x - alpha / (np.sqrt(s) + eps) * g  # per-dimension scaled update
        path.append(x.copy())

    return x, np.array(path)

# -- Running example: f(x) = x1^2 + 10*x2^2
#    Gradient: [2*x1, 20*x2]
A = np.diag([2.0, 20.0])
grad_f = lambda x: A @ x

x0 = np.array([3.5, 3.5])
```

5.6 RMSProp — Fixing AdaGrad's Decaying Step I

The core idea. AdaGrad's problem is that $s_i^{(k)}$ accumulates all squared gradients from the very beginning, so it grows without bound. RMSProp (Tieleman and Hinton, 2012) replaces this *arithmetic sum* with an **exponential moving average** (EMA), which effectively "forgets" old squared gradients. This allows the effective step size to stabilise rather than decay monotonically.

5.6 RMSProp — Fixing AdaGrad's Decaying Step II

RMSProp Update

Instead of summing all past squared gradients, compute their exponential moving average:

$$\mathbf{s}^{(k)} = \gamma\, \mathbf{s}^{(k-1)} + (1-\gamma)\, \mathbf{g}^{(k)} \odot \mathbf{g}^{(k)}$$

Then apply per-dimension scaling:

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} - \frac{\alpha}{\sqrt{\mathbf{s}^{(k)}} + \varepsilon} \odot \mathbf{g}^{(k)}$$

Symbol explanations:

- $\mathbf{s}^{(k)}$ (bold s superscript k): the EMA of squared gradients; this is a *vector* with one entry per dimension.
- γ (gamma): the EMA decay rate, typically 0.9. Values closer to 1 give a longer memory; values closer to 0 give a shorter memory.
- $\odot$ (Hadamard product): elementwise multiplication of two vectors.
- α (alpha): the global step factor, typically 0.001.
- ε (epsilon) $\approx 10^{-8}$: prevents division by zero; also applied *inside* the square root for stability.

5.6 RMSProp — Python Implementation

```python
import numpy as np

def rmsprop(grad_f, x, alpha=0.001, gamma=0.9,
            eps=1e-8, n_iter=500):
    """
    RMSProp: exponential moving average of squared gradients.
    grad_f : gradient function
    x      : initial point
    alpha  : global learning rate (typically 0.001)
    gamma  : EMA decay rate (typically 0.9; controls memory length)
    eps    : small constant for numerical stability
    """
    s = np.zeros_like(x)    # EMA of squared gradients, init at zero
    path = [x.copy()]

    for _ in range(n_iter):
        g = grad_f(x)
        # EMA update: keep fraction gamma of old s, add (1-gamma) of new g^2
        s = gamma * s + (1 - gamma) * g**2
        # Scale update by 1 / (sqrt(s) + eps) elementwise
        x = x - alpha / (np.sqrt(s) + eps) * g
        path.append(x.copy())

    return x, np.array(path)

# -- Example: f(x) = x1^2 + 10*x2^2
#    This is ill-conditioned (ratio 10:1); adaptive methods handle it better.
```

5.7 Adadelta — No Global Learning Rate I

Motivation. RMSProp still requires a global learning rate α (alpha) that must be manually tuned. Adadelta (Zeiler, 2012) takes the adaptive idea one step further by eliminating α entirely. Instead of scaling by a fixed α, it uses the ratio of two exponential moving averages to automatically determine an appropriate step size.

Units-based motivation. The gradient has units of $\partial f / \partial x_i$ (change in objective per unit change in x_i). The ideal update Δx_i should have units of x_i, not gradient units. Adadelta achieves this unit invariance by forming a ratio that cancels out the gradient units.

5.7 Adadelta — No Global Learning Rate II

Adadelta Update

Maintain two exponential moving averages:

$$\mathbf{s}^{(k)} = \gamma_s \, \mathbf{s}^{(k-1)} + (1 - \gamma_s) \, \mathbf{g}^{(k)} \odot \mathbf{g}^{(k)} \qquad \text{(EMA of squared gradients)}$$

$$\Delta \mathbf{x}^{(k)} = -\frac{\sqrt{\mathbf{u}^{(k-1)} + \varepsilon}}{\sqrt{\mathbf{s}^{(k)} + \varepsilon}} \odot \mathbf{g}^{(k)} \qquad \text{(update step)}$$

$$\mathbf{u}^{(k)} = \gamma_x \, \mathbf{u}^{(k-1)} + (1 - \gamma_x) \, \Delta \mathbf{x}^{(k)} \odot \Delta \mathbf{x}^{(k)} \qquad \text{(EMA of squared updates)}$$

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \Delta \mathbf{x}^{(k)} \qquad \text{(parameter update)}$$

5.7 Adadelta — No Global Learning Rate III

Symbol explanations:

- $\mathbf{s}^{(k)}$ (bold s superscript k): EMA of squared gradients (same as in RMSProp).
- $\mathbf{u}^{(k)}$ (bold u superscript k): EMA of squared *updates* $\Delta\mathbf{x}$. This tracks the recent history of how large the steps have been.
- γ_s (gamma sub s): decay rate for the gradient EMA, typically 0.95.
- γ_x (gamma sub x): decay rate for the update EMA, typically 0.95.
- ε (epsilon): small constant added to *both* numerator and denominator; adding it to the numerator ensures the very first step is well-defined (since $\mathbf{u}^{(0)} = \mathbf{0}$).
- $\Delta\mathbf{x}^{(k)}$ (Delta x superscript k): the update vector applied to $\mathbf{x}$ at step k; "delta" denotes a change.

5.7 Adadelta — No Global Learning Rate IV

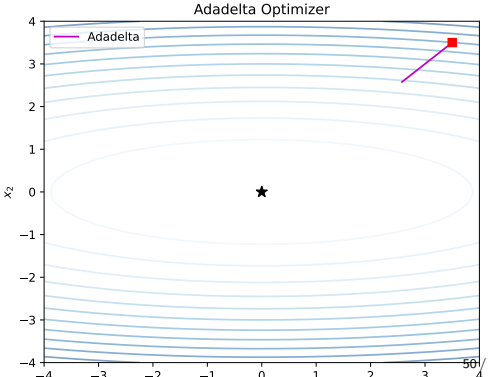
How to Read This

The numerator $\sqrt{\mathbf{u}^{(k-1)}}$ has units of update magnitude (x units) and the denominator $\sqrt{\mathbf{s}^{(k)}}$ has units of gradient magnitude. Their ratio therefore has units of x/gradient, which is exactly the unit of a (approximate) Hessian inverse applied to a gradient. Adadelta thus mimics a Newton-like step without ever computing the Hessian.

Typical Hyperparameters

$\gamma_s = 0.95, \quad \gamma_x = 0.95, \quad \varepsilon = 10^{-6}$.
Note: there is *no* α to tune. This is Adadelta's main practical advantage in settings where learning rate search is expensive.

Practical note. In practice, Adadelta converges reliably but can be slower than Adam in the early iterations, because the numerator EMA **u** starts at zero and takes time to warm up.



5.7 Adadelta — No Global Learning Rate V

Key Takeaway

Adadelta eliminates the need to set a global learning rate, making it useful for practitioners who want robust "set and forget" optimisation. The price paid is two EMA hyperparameters (γ_s and γ_x) instead of one, though these are far less sensitive than the learning rate.

5.7 Adadelta — Python Implementation

```python
import numpy as np

def adadelta(grad_f, x, gamma_s=0.95, gamma_x=0.95,
             eps=1e-6, n_iter=500):
    """
    Adadelta: eliminates the global learning rate entirely.
    grad_f  : gradient function
    x       : initial point
    gamma_s : EMA decay for squared gradients (typical: 0.95)
    gamma_x : EMA decay for squared updates (typical: 0.95)
    eps     : small constant added to BOTH numerator AND denominator
              (critical: numerator eps ensures first step works when u=0)
    """
    s = np.zeros_like(x)   # EMA of squared gradients
    u = np.zeros_like(x)   # EMA of squared updates (starts at 0)
    path = [x.copy()]

    for _ in range(n_iter):
        g = grad_f(x)
        # Update EMA of squared gradients
        s = gamma_s * s + (1 - gamma_s) * g**2
        # Compute update step: ratio of RMS(updates) to RMS(gradients)
        dx = -(np.sqrt(u + eps) / np.sqrt(s + eps)) * g
        # Update EMA of squared updates (using the step just taken)
        u = gamma_x * u + (1 - gamma_x) * dx**2
        x = x + dx
        path.append(x.copy())
```

5.8 Adam — Adaptive Moment Estimation I

What Adam is. Adam (Kingma and Ba, 2014) stands for **Ada**ptive **M**oment estimation. It combines two ideas we have already seen:

1. **Momentum**: maintain an EMA of the gradient itself (the first moment, or mean).
2. **RMSProp**: maintain an EMA of the squared gradient (the second moment, or uncentered variance).

Adam also adds **bias correction**, which is crucial during the first few iterations.

5.8 Adam — Adaptive Moment Estimation II

Adam Update

Track biased first moment $\mathbf{v}$ (momentum) and biased second moment $\mathbf{s}$ (squared gradient EMA):

$$\mathbf{v}^{(k)} = \gamma_v \, \mathbf{v}^{(k-1)} + (1 - \gamma_v) \, \mathbf{g}^{(k)}$$

$$\mathbf{s}^{(k)} = \gamma_s \, \mathbf{s}^{(k-1)} + (1 - \gamma_s) \, \mathbf{g}^{(k)} \odot \mathbf{g}^{(k)}$$

Apply **bias correction** to both:

$$\hat{\mathbf{v}}^{(k)} = \frac{\mathbf{v}^{(k)}}{1 - \gamma_v^k}, \qquad \hat{\mathbf{s}}^{(k)} = \frac{\mathbf{s}^{(k)}}{1 - \gamma_s^k}$$

Finally, update the parameters:

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} - \alpha \, \frac{\hat{\mathbf{v}}^{(k)}}{\sqrt{\hat{\mathbf{s}}^{(k)}} + \varepsilon}$$

5.8 Adam — Adaptive Moment Estimation III

Symbol explanations:

- $\mathbf{v}^{(k)}$ (bold v superscript k): the biased first moment estimate (EMA of gradients), analogous to the velocity in momentum. It gives Adam a momentum-like property.
- $\mathbf{s}^{(k)}$ (bold s superscript k): the biased second moment estimate (EMA of squared gradients), analogous to the **s** in RMSProp.
- γ_v (gamma sub v): decay rate for the first moment, typically 0.9. A value of 0.9 means each step blends 10% of the new gradient with 90% of the accumulated average.
- γ_s (gamma sub s): decay rate for the second moment, typically 0.999. This very high value means the second moment is a very smooth average over a long history.
- $\hat{\mathbf{v}}^{(k)}$ (v-hat superscript k): the bias-corrected first moment. The "hat" indicates a corrected estimate.
- $\hat{\mathbf{s}}^{(k)}$ (s-hat superscript k): the bias-corrected second moment.
- γ_v^k (gamma sub v to the power k): gamma sub v raised to the k-th power. This quantity decreases toward 0 as k grows.
- α (alpha): global learning rate, typically 0.001.

5.8 Adam — Adaptive Moment Estimation IV

- ε (epsilon) $\approx 10^{-8}$: prevents division by zero.

Default Hyperparameters

Parameter	Symbol	Default value
Learning rate	α (alpha)	0.001
First moment decay	γ_v (gamma sub v)	0.9
Second moment decay	γ_s (gamma sub s)	0.999
Stability constant	ε (epsilon)	10^{-8}
Initial first moment	$\mathbf{v}^{(0)}$	$\mathbf{0}$
Initial second moment	$\mathbf{s}^{(0)}$	$\mathbf{0}$

5.8 Adam — Adaptive Moment Estimation V

Why bias correction is necessary. Both $\mathbf{v}$ and $\mathbf{s}$ are initialised at zero. After the very first step:

$$\mathbf{v}^{(1)} = (1 - \gamma_v)\, \mathbf{g}^{(1)}$$

With $\gamma_v = 0.9$, this gives $\mathbf{v}^{(1)} = 0.1\, \mathbf{g}^{(1)}$, which is only 10% of the true gradient. Without correction, the effective learning rate would be ten times smaller than intended during the first several iterations. Bias correction divides by $1 - \gamma_v^k$:

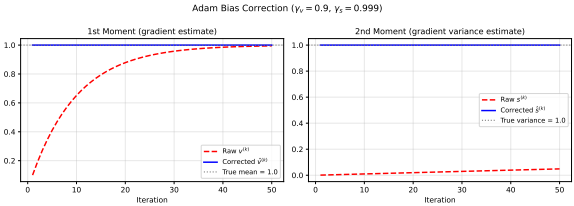
$$\hat{\mathbf{v}}^{(k)} = \frac{\mathbf{v}^{(k)}}{1 - \gamma_v^k}$$

At $k = 1$: $1 - 0.9^1 = 0.1$, so $\hat{\mathbf{v}}^{(1)} = \mathbf{g}^{(1)}$ (correctly scaled). As $k \to \infty$: $1 - \gamma_v^k \to 1$, so the correction factor disappears.

How to Read This

Bias correction is only important in the *first few iterations*. Without it, the algorithm starts much more cautiously than intended. With it, Adam behaves consistently from the very first step onward.

5.8 Adam — Adaptive Moment Estimation VI



Without bias correction (dashed), the effective moments are severely underestimated early in training. With correction (solid), they track the true values from the start.

Adam Combines Three Ideas

- **Momentum** (via **v**): accelerates convergence in consistent gradient directions.
- **Adaptive per-dimension scaling** (via **s**): handles dimensions with different gradient magnitudes automatically.
- **Bias correction**: ensures correct behaviour from the very first iteration.

Together these make Adam a robust default for a wide range of machine learning problems, especially deep learning.

5.8 Adam — Adaptive Moment Estimation VII

Key Takeaway

Adam is the most widely used optimizer in deep learning today because it works well with minimal hyperparameter tuning. The default hyperparameters ($\alpha = 0.001, \gamma_v = 0.9, \gamma_s = 0.999$) work surprisingly well across a broad range of models and datasets.

5.8 Adam — Python Implementation

```python
import numpy as np

def adam(grad_f, x, alpha=0.001, gamma_v=0.9, gamma_s=0.999,
         eps=1e-8, n_iter=1000):
    """
    Adam: Adaptive Moment Estimation (Kingma & Ba, 2014).
    grad_f  : gradient function
    x       : initial point
    alpha   : global learning rate (default 0.001; rarely needs tuning)
    gamma_v : first moment (momentum) decay, default 0.9
    gamma_s : second moment (squared grad) decay, default 0.999
    eps     : stability constant, prevents division by zero
    """
    v = np.zeros_like(x)   # biased first moment (momentum), init zero
    s = np.zeros_like(x)   # biased second moment (sq. grad), init zero
    path = [x.copy()]

    for k in range(1, n_iter + 1):
        g = grad_f(x)
        v = gamma_v * v + (1 - gamma_v) * g         # biased first moment
        s = gamma_s * s + (1 - gamma_s) * g**2      # biased second moment
        # Bias correction: divide by (1 - gamma^k)
        v_hat = v / (1 - gamma_v**k)                # corrected first moment
        s_hat = s / (1 - gamma_s**k)                # corrected second moment
        # Parameter update: scale gradient by first moment / sqrt(second moment)
        x = x - alpha * v_hat / (np.sqrt(s_hat) + eps)
        path.append(x.copy())
```

5.8 Adam — Step-by-Step Worked Example I

Let us trace through the first few Adam iterations manually to build intuition. We minimise $f(\mathbf{x}) = x_1^2 + 10x_2^2$ starting from $\mathbf{x}^{(1)} = [3.5,\ 3.5]$ with default hyperparameters $\alpha = 0.001$, $\gamma_v = 0.9$, $\gamma_s = 0.999$.

Iteration $k = 1$

Gradient: $\mathbf{g}^{(1)} = [2 \times 3.5,\ 20 \times 3.5] = [7.0,\ 70.0]$.
First moment: $\mathbf{v}^{(1)} = 0.9 \times [0, 0] + 0.1 \times [7.0, 70.0] = [0.7,\ 7.0]$.
Second moment: $\mathbf{s}^{(1)} = 0.999 \times [0, 0] + 0.001 \times [49,\ 4900] = [0.049,\ 4.9]$.
Bias correction (at $k = 1$, denominator $= 1 - 0.9^1 = 0.1$ and $1 - 0.999^1 = 0.001$):
$\hat{\mathbf{v}}^{(1)} = [0.7/0.1,\ 7.0/0.1] = [7.0,\ 70.0]$. $\hat{\mathbf{s}}^{(1)} = [0.049/0.001,\ 4.9/0.001] = [49,\ 4900]$.
Update: $\mathbf{x}^{(2)} = [3.5, 3.5] - 0.001 \times \left[\frac{7.0}{\sqrt{49}+\varepsilon},\ \frac{70.0}{\sqrt{4900}+\varepsilon}\right] = [3.5, 3.5] - 0.001 \times [1,\ 1] = [3.499,\ 3.499]$.
Both dimensions move by exactly $\alpha = 0.001$ per step, because the bias-corrected moments yield $\hat{v}_i/\sqrt{\hat{s}_i} \approx 1$.

5.8 Adam — Step-by-Step Worked Example II

What Happens Next

As iterations continue:

- The bias correction factor $(1 - \gamma^k)$ grows toward 1, so the correction becomes less and less important.
- The second moment $\hat{s}_i$ adapts to reflect the recent gradient magnitude in each dimension, so dimension 2 (with gradient $20x_2$, much larger than dimension 1's $2x_1$) takes proportionally smaller steps.
- The first moment $\hat{v}$ smooths out any gradient noise, acting like momentum.

The net effect is that both x_1 and x_2 converge to 0, with x_2 converging in a similar number of steps as x_1 despite its much larger gradient, because Adam automatically compensates.

5.8 Adam — Step-Trace Python Code

```python
import numpy as np

def adam_trace(alpha=0.001, gamma_v=0.9,
               gamma_s=0.999, eps=1e-8):
    """Print first 5 Adam iterates for f(x) = x1^2 + 10*x2^2."""
    x = np.array([3.5, 3.5])
    v = np.zeros(2)
    s = np.zeros(2)
    A = np.diag([2.0, 20.0])   # gradient = A @ x

    for k in range(1, 6):
        g = A @ x                          # gradient
        v = gamma_v * v + (1 - gamma_v) * g
        s = gamma_s * s + (1 - gamma_s) * g**2
        v_hat = v / (1 - gamma_v**k)       # bias-corrected first moment
        s_hat = s / (1 - gamma_s**k)       # bias-corrected second moment
        step = alpha * v_hat / (np.sqrt(s_hat) + eps)
        x = x - step
        print(f"k={k}: x={x.round(4)}, "
              f"|g|={np.linalg.norm(g):.3f}, "
              f"step={step.round(5)}")

adam_trace()
# Expected output:
# k=1: x=[3.499 3.499], |g|=70.711, step=[0.001 0.001]
# k=2: x=[3.498 3.498], |g|=70.686, ...
# Both dimensions move together despite 10:1 gradient ratio!
```

5.9 Hypergradient Descent — Learning the Learning Rate I

The meta-problem. Every gradient-based method requires a step factor α (alpha), and choosing it is perhaps the most important and frustrating hyperparameter decision in practice. Too small and convergence is painfully slow; too large and the algorithm diverges. Can we automate this choice?

The idea. Treat α as an additional variable and apply gradient descent *to it*. This is called *hypergradient descent*: the term "hyper" indicates we are taking a gradient with respect to a hyperparameter.

Deriving the hypergradient. For ordinary gradient descent $\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} - \alpha^{(k)}\mathbf{g}^{(k)}$, we want to compute how the function value at the *next* iterate depends on the current step size $\alpha^{(k)}$:

$$\frac{\partial f\left(\mathbf{x}^{(k+1)}\right)}{\partial \alpha^{(k)}} = \left(\mathbf{g}^{(k+1)}\right)^{\top} \frac{\partial \mathbf{x}^{(k+1)}}{\partial \alpha^{(k)}} = \left(\mathbf{g}^{(k+1)}\right)^{\top} \frac{\partial}{\partial \alpha^{(k)}} \left(\mathbf{x}^{(k)} - \alpha^{(k)}\mathbf{g}^{(k)}\right)$$

Since $\mathbf{x}^{(k)}$ and $\mathbf{g}^{(k)}$ do not depend on $\alpha^{(k)}$, the partial derivative of the bracket is simply $-\mathbf{g}^{(k)}$:

5.9 Hypergradient Descent — Learning the Learning Rate II

Hypergradient of the Step Size

$$\frac{\partial f(\mathbf{x}^{(k+1)})}{\partial \alpha^{(k)}} = -\left(\mathbf{g}^{(k+1)}\right)^{\top} \mathbf{g}^{(k)}$$

This is the negative dot product of the current and previous gradients. Remarkably, computing it costs no extra function evaluations — we already have both $\mathbf{g}^{(k)}$ and $\mathbf{g}^{(k+1)}$ from adjacent gradient descent steps.

5.9 Hypergradient Descent — Learning the Learning Rate III

The step-size update rule. Applying gradient descent to α with a "hyper step factor" μ (mu):

Hypergradient Step-Size Update

$$\alpha^{(k+1)} = \alpha^{(k)} - \mu \cdot \frac{\partial f(\mathbf{x}^{(k+1)})}{\partial \alpha^{(k)}} = \alpha^{(k)} + \mu \, \mathbf{g}^{(k+1)} \cdot \mathbf{g}^{(k)}$$

Here μ (mu) is the *meta-learning rate*: a new hyperparameter that controls how quickly α adapts. It must be small (e.g. 10^{-6} to 10^{-4}) to keep the adaptation stable.

5.9 Hypergradient Descent — Learning the Learning Rate IV

How to Read This

The dot product $\mathbf{g}^{(k+1)} \cdot \mathbf{g}^{(k)}$ measures the angle between successive gradients:

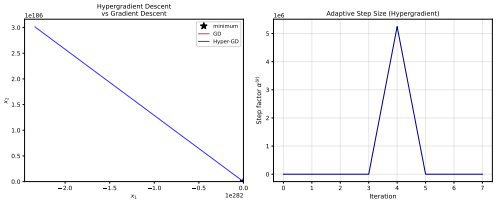
- If successive gradients point in the *same* direction (dot product > 0): the step size was conservative; increasing α is safe.
- If successive gradients point in *opposite* directions (dot product < 0): we overshot; decreasing α prevents divergence.

The algorithm is thus self-correcting: it automatically detects oscillation (the telltale sign of too-large α) and reduces the step size.

5.9 Hypergradient Descent — Learning the Learning Rate V

Worked Intuition

Consider a 1D function with minimum at $x^* = 0$.
Starting at $x^{(1)} = 1$, $\alpha^{(1)} = 0.5$:
Step 1: $g^{(1)} = f'(1) > 0$ (gradient points right, away from minimum). Update: $x^{(2)} = 1 - 0.5g^{(1)}$.
Step 2: If $x^{(2)} > 0$, then $g^{(2)} > 0$ again (same direction). Dot product $g^{(2)} \cdot g^{(1)} > 0$: both gradients point right, so α increases.
If $x^{(2)} < 0$ (overshot), $g^{(2)} < 0$ (gradient now points left), so dot product < 0 and α decreases automatically.



Left: hypergradient GD (solid) vs. standard GD (dashed). Right: the step size α adapts over iterations, growing when progress is consistent and shrinking when oscillation is detected.

Extension to Nesterov. The hypergradient idea extends to Nesterov momentum: the step size α is updated using the dot product of successive *lookahead gradients* $\nabla f(\mathbf{x}^{(k)} + \beta \mathbf{v}^{(k)})$, yielding Hyper-Nesterov (Algorithm 5.10 in the book).

5.9 Hypergradient Descent — Learning the Learning Rate VI

Key Takeaway

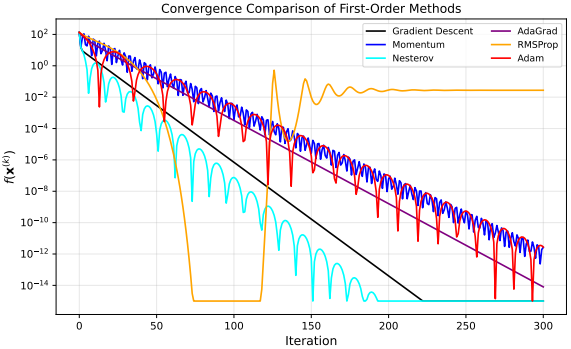
Hypergradient descent automatically adapts the learning rate during optimisation at essentially no extra cost (one extra dot product per step). The trade-off is that it introduces a new meta-hyperparameter μ (mu), though μ is typically much easier to set than α.

5.9 Hypergradient Descent — Python Implementation

```python
import numpy as np

def hypergradient_descent(grad_f, x, alpha0=0.001,
                          mu=1e-6, n_iter=500):
    """
    Hypergradient descent: gradient descent on the step factor alpha.
    grad_f : gradient function
    x      : initial point
    alpha0 : initial step factor (alpha will adapt from here)
    mu     : meta-learning rate (how fast alpha adapts); keep small!
    """
    alpha = alpha0
    g_prev = np.zeros_like(x)   # previous gradient (zero at start)
    path = [x.copy()]

    for _ in range(n_iter):
        g = grad_f(x)
        # Update alpha: increase if gradients consistent, decrease if oscillating
        alpha = alpha + mu * np.dot(g, g_prev)   # hypergradient update
        alpha = max(alpha, 1e-10)                # keep alpha positive
        g_prev = g.copy()                        # save for next step
        x = x - alpha * g
        path.append(x.copy())

    return x, np.array(path)

def hyper_nesterov(grad_f, x, alpha0=0.001, beta=0.9,
```

Convergence Comparison of All Methods I

Now that we have seen all nine methods, it is useful to compare them side by side on the same test function to understand their relative strengths and weaknesses. The figure below shows convergence on the ill-conditioned quadratic $f(\mathbf{x}) = x_1^2 + 10x_2^2$, starting from $\mathbf{x}^{(0)} = [3.5,\ 3.5]$.



All methods applied to $f(\mathbf{x}) = x_1^2 + 10x_2^2$, starting from $\mathbf{x}^{(0)} = [3.5,\ 3.5]$. The vertical axis shows $f(\mathbf{x}^{(k)})$ on a logarithmic scale; a steeper downward slope means faster convergence.

Method	Convergence	Notes
GD	$O(1/k)$	Baseline
Conjugate GD	n exact steps	For quadratics
Momentum	$O(1/k)$	Faster in practice
Nesterov	$O(1/k^2)$	Optimal for convex f
AdaGrad	$O(1/\sqrt{k})$	Best for sparse grads
RMSProp	Adaptive	Robust for deep nets
Adadelta	Adaptive	No α needed
Adam	Adaptive	Deep learning default
Hypergradient	Adaptive	Learns α online

Convergence Comparison of All Methods II

Reading the convergence rates. The rate $O(1/k)$ (read "big-Oh of one over k") means the error after k iterations is at most some constant divided by k. To halve the error, you need twice as many iterations. The rate $O(1/k^2)$ means the error decreases with the *square* of the number of iterations: to halve the error, you only need $\sqrt{2} \approx 1.41$ times as many iterations. This is why Nesterov's $O(1/k^2)$ is called "accelerated."

Practical takeaways.

- For **deep learning**, Adam is the de-facto standard because its default hyperparameters work well across a huge variety of architectures and loss functions.
- For **smooth convex problems** (e.g. logistic regression, support vector machines), Nesterov momentum achieves the theoretically optimal rate and is often the best practical choice.
- For **quadratic problems**, conjugate gradient converges in exactly n steps; no other first-order method can beat this.
- When **learning rate tuning is expensive**, Adadelta or hypergradient descent can save significant effort.

Convergence Comparison of All Methods III

- For **sparse features** (text, recommendation systems), AdaGrad remains competitive because its per-dimension scaling perfectly matches the varying frequency of features.

Key Takeaway

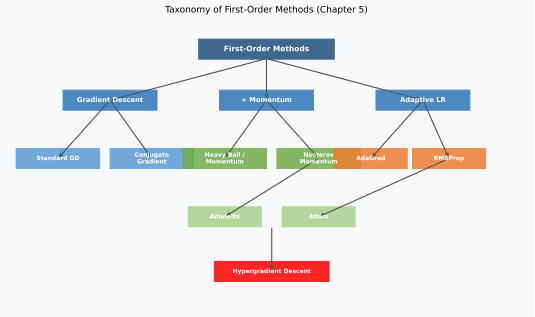
No single first-order method is best for all problems. Understanding the structure of your problem — smooth or non-smooth, convex or non-convex, sparse or dense gradients, short run or long run — is the key to choosing the right optimizer.

5.10 Summary I

This chapter developed nine first-order methods, each one addressing a specific limitation of the previous. Here is a concise summary of the key ideas:

Summary of All Methods

- **Gradient descent** moves in the direction of steepest descent $-\nabla f / \|\nabla f\|$. Simple, but zigzags on ill-conditioned functions.
- **Conjugate gradient** avoids the zigzag by maintaining directions that are mutually conjugate (A-orthogonal), converging in n steps on quadratics.
- **Momentum** accumulates a velocity vector to accelerate progress, damping out oscillations and accelerating in consistent-gradient directions.
- **Nesterov momentum** evaluates the gradient at the anticipated next position (lookahead), achieving the optimal $O(1/k^2)$ rate for convex f.



Taxonomy of first-order methods, showing how each method extends or modifies the previous.

The Unifying Theme

Every method in this chapter starts from the same basic idea — move in the direction that decreases f — and adds one more piece of information or structure to do so more effectively. Understanding this progression is the most important takeaway.

Exercise 5.1: Normalised Gradient Descent Step

For $f(\mathbf{x}) = x_1 x_2^2$, at $\mathbf{x}^{(k)} = [1, 2]$, compute the normalised gradient descent direction $\mathbf{d}^{(k)}$.

Solution. We first compute the gradient. The partial derivatives are:

$$\frac{\partial f}{\partial x_1} = x_2^2 = 4, \qquad \frac{\partial f}{\partial x_2} = 2x_1 x_2 = 4.$$

So $\nabla f(1, 2) = [4,\ 4]$ and the unnormalised descent direction is $-[4, 4]$. Normalising:

$$\mathbf{d}^{(k)} = -\frac{[4,4]}{\|[4,4]\|} = -\frac{[4,4]}{\sqrt{32}} = \left[-\frac{1}{\sqrt{2}}, -\frac{1}{\sqrt{2}}\right] \approx [-0.707,\ -0.707].$$

The step moves equally in both the $-x_1$ and $-x_2$ directions.

Exercise 5.3: Effect of Too-Large Step Size

Apply gradient descent with $\alpha = 1$ to $f(x) = x^4$, starting at $x^{(1)} = 1$. What happens?

Solution. The gradient is $f'(x) = 4x^3$.

$$x^{(2)} = 1 - 1 \times f'(1) = 1 - 4 = -3$$
$$x^{(3)} = -3 - 1 \times f'(-3) = -3 - 4(-27) = -3 + 108 = 105$$

The iterates diverge: each step overshoots the minimum at $x = 0$ by a larger and larger amount. The step size $\alpha = 1$ is far too large for this function. A step size of $\alpha \leq 0.05$ would be stable.

Exercise 5.8: Conjugate Gradient on a Quadratic

Apply CG to $f(x, y) = x^2 + xy + y^2 + 5$, starting at $(1, 1)$. How quickly does it converge?

Solution. The gradient is $\nabla f(x, y) = [2x + y,\ x + 2y]$. At $(1, 1)$: $\nabla f = [3, 3]$, so the first CG direction is $\mathbf{d}^{(1)} = -[3, 3]$.

The Hessian (second derivative matrix) is $\mathbf{H} = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$, which is positive definite (both eigenvalues positive: 1 and 3). Since $n = 2$, CG is guaranteed to converge in at most **2** steps to the exact minimum. The minimum is at $(0, 0)$, where $f(0, 0) = 5$ (the constant term).

Exercise 5.6: Why Is Nesterov Better Than Standard Momentum?

Explain the improvement Nesterov momentum makes over standard momentum.

5.11 Selected Exercises with Solutions IV

Solution. In standard momentum, the gradient is evaluated at the *current* position $\mathbf{x}^{(k)}$ before the velocity $\mathbf{v}^{(k)}$ is applied. In Nesterov momentum, the gradient is evaluated at the *lookahead* position $\mathbf{x}^{(k)} + \beta\mathbf{v}^{(k)}$, which is where the algorithm will actually move to if no gradient correction were applied. By evaluating the gradient at the destination rather than the starting point, Nesterov obtains a more accurate estimate of the gradient for the current step. This prevents overshooting because the gradient "sees" that it is approaching the minimum and begins slowing down earlier. The result is the provably optimal $O(1/k^2)$ convergence rate for smooth convex functions, compared to $O(1/k)$ for standard gradient descent.

Exercise 5.10: Heavy Ball Equivalence

Show that the heavy ball iterates are identical to those of the momentum method.

Solution. Define the velocity as $\mathbf{v}^{(k)} = \mathbf{x}^{(k)} - \mathbf{x}^{(k-1)}$ (the change in position from the previous to the current step). The heavy ball update is:

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} - \alpha \mathbf{g}^{(k)} + \beta \mathbf{v}^{(k)} = \mathbf{x}^{(k)} - \alpha \mathbf{g}^{(k)} + \beta \left(\mathbf{x}^{(k)} - \mathbf{x}^{(k-1)} \right).$$

The implied velocity at step $k+1$ is:

$$\mathbf{v}^{(k+1)} = \mathbf{x}^{(k+1)} - \mathbf{x}^{(k)} = -\alpha \mathbf{g}^{(k)} + \beta \mathbf{v}^{(k)}.$$

This is exactly the momentum update rule $\mathbf{v}^{(k+1)} = \beta \mathbf{v}^{(k)} - \alpha \mathbf{g}^{(k)}$. Therefore, the two formulations produce identical iterates; they are just different ways of writing the same algorithm.

Further Reading and Resources I

Original Papers

- **Conjugate Gradient**: R. Fletcher & C. M. Reeves, "Function Minimization by Conjugate Gradients," *The Computer Journal*, 1964.
- **Momentum (Heavy Ball)**: B. T. Polyak, "Some Methods of Speeding Up the Convergence of Iteration Methods," 1964.
- **Nesterov**: Y. Nesterov, "A Method of Solving a Convex Programming Problem with Convergence Rate $O(1/k^2)$," 1983.
- **AdaGrad**: J. Duchi, E. Hazan, & Y. Singer, "Adaptive Subgradient Methods," JMLR, 2011.
- **RMSProp**: T. Tieleman & G. Hinton, Lecture 6.5, Coursera, 2012 (unpublished).
- **Adadelta**: M. D. Zeiler, "ADADELTA: An Adaptive Learning Rate Method," 2012.
- **Adam**: D. P. Kingma & J. Ba, "Adam: A Method for Stochastic Optimization," ICLR, 2015.

Python Ecosystem

All methods in this chapter are implemented in modern deep learning frameworks:

Method	Python Package / Function
All methods	`torch.optim` (PyTorch)
Adam, etc.	`tf.keras.optimizers`
L-BFGS (CG-like)	`scipy.optimize.minimize`
Custom	`numpy` (as shown above)

Final Takeaway

The algorithms in this chapter form the engine of modern machine learning. Knowing how they work — not just how to call them — makes you a much better practitioner.

Further Reading — scipy Example

The `scipy.optimize` library provides a well-tested implementation of the conjugate gradient method that can be applied to any smooth objective function. Using it requires only providing f and its gradient ∇f:

```
from scipy.optimize import minimize
import numpy as np

f     = lambda x: x[0]**2 + 10*x[1]**2
gradf = lambda x: np.array([2*x[0], 20*x[1]])

# Use scipy's conjugate-gradient method (method='CG')
res = minimize(f, [3.5, 3.5],
               jac=gradf,
               method='CG')
print(res.x)       # approximately [0., 0.]
print(res.nit)     # number of CG iterations used
```

Listing 11: scipy Conjugate Gradient Example

The `minimize` function returns a result object `res` with fields including `res.x` (the optimal point found), `res.fun` (the minimum function value), and `res.nit` (number of iterations). For well-conditioned quadratics like this example, `method='CG'` typically needs very few iterations.